Solution: Remove Nodes From Linked List

Problem Statement

Given the head node of a singly linked list, modify the list such that any node that has a node with a greater value to its right gets removed. The function should return the head of the modified list.

Examples:

1. Input: 5 -> 3 -> 7 -> 4 -> 2 -> 1
   Output: 7 -> 4 -> 2 -> 1
   Explanation: 5 and 3 are removed as they have nodes with larger values to their right.

2. Input: 1 -> 2 -> 3 -> 4 -> 5
   Output: 5
   Explanation: 1, 2, 3, and 4 are removed as they have nodes with larger values to their right.

3. Input: 5 -> 4 -> 3 -> 2 -> 1
   Output: 5 -> 4 -> 3 -> 2 -> 1
   Explanation: None of the nodes are removed as none of them have nodes with larger values to their right.

Solution

We can the monotonic stack strategy to keep track of the highest-valued nodes in the linked list, ensuring that any node with a higher value to its right gets removed.

Starting from the head of the list, we will traverse each node. At each node, we will check the value of the node against the node at the top of the stack. If the current node has a higher value, we will pop the top value from the stack. We will keep repeating this until we encounter a node with a higher value or the stack is empty. Then, the current node is pushed onto the stack. This process ensures that the stack only contains nodes in decreasing order from bottom to top.

Detailed Walkthrough:

1. Create an empty stack, `stack`.
2. Initialize `cur` with the `head` of the linked list.
3. Enter a loop that continues until `cur` becomes `null`:
   a. Enter an inner loop that continues as long as `cur`'s value is greater than the value of the node at the top of `stack`:
      - Pop the top node from `stack`.

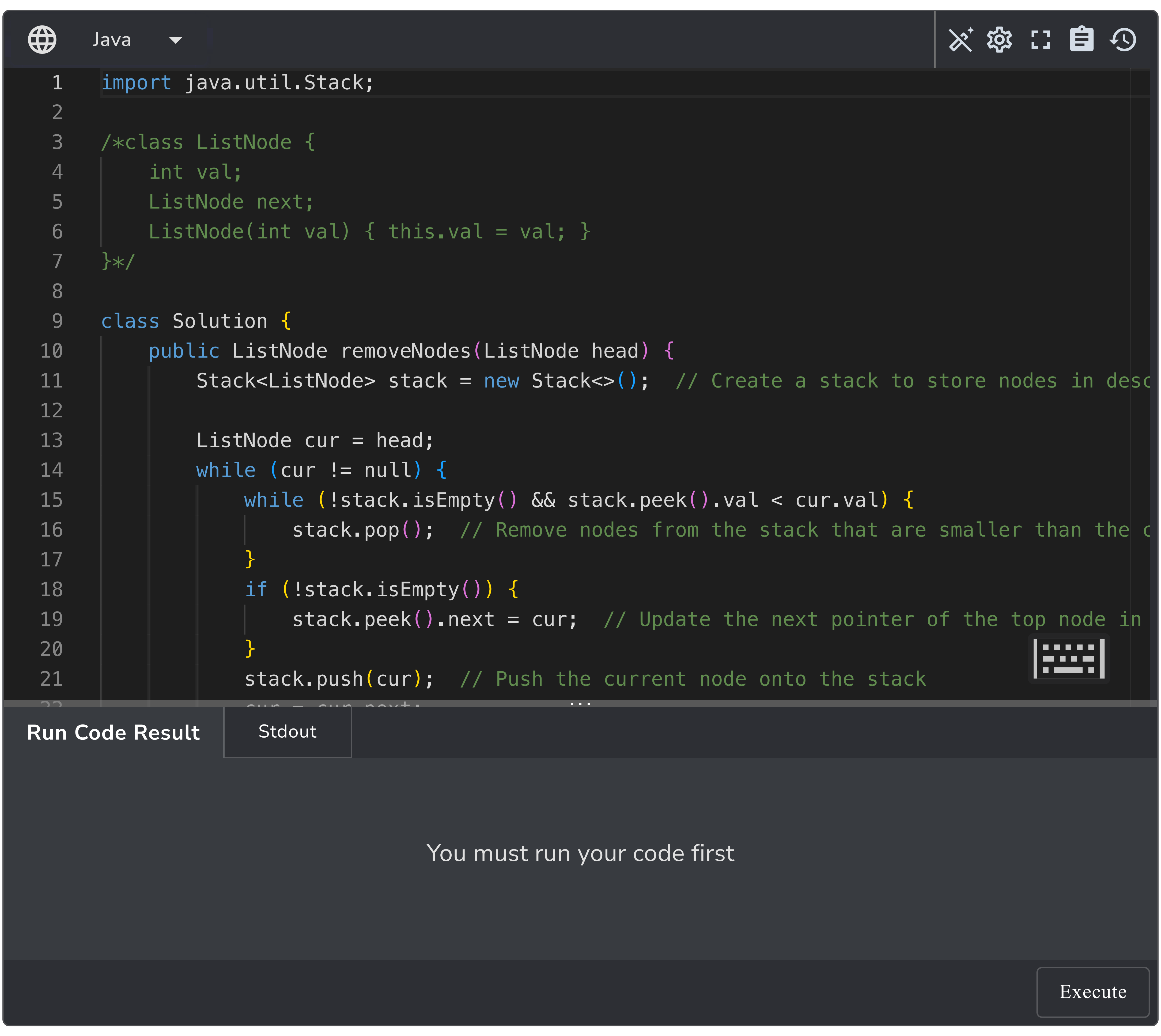
   b. If `stack` is not empty, set the `next` of the node at the top of `stack` to `cur`.

   c. Push `cur` onto `stack`.

   d. Move `cur` to the next node in the linked list.
4. After exiting the loop, return the bottom node of `stack` as the head of the modified linked list. If `stack` is empty, return `null`.

Code

Here is how we can implement this algorithm:



```java
import java.util.Stack;

/*class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
}*/

class Solution {
    public ListNode removeNodes(ListNode head) {
        Stack<ListNode> stack = new Stack<>();  // Create a stack to store nodes in desc

        ListNode cur = head;
        while (cur != null) {
            while (!stack.isEmpty() && stack.peek().val < cur.val) {
                stack.pop();  // Remove nodes from the stack that are smaller than the c
            }
            if (!stack.isEmpty()) {
                stack.peek().next = cur;  // Update the next pointer of the top node in
            }
            stack.push(cur);  // Push the current node onto the stack
```

You must run your code first

Time and Space Complexity

The time complexity of this algorithm is $O(n)$, where n is the number of nodes in the linked list. This is because every node is visited once while traversing the list. The inner loop may seem to increase the time complexity, but in fact, it doesn't, because each node is pushed and popped from the stack at most once, so the total operations are still proportional to n.

The space complexity is also $O(n)$. This is due to the extra space required for the stack. In the worst-case scenario (when the list is sorted in ascending order), all the nodes will be pushed onto the stack, requiring n space.

Therefore, we say the algorithm is linear in both time and space complexity.

← Previous Next →
Remove Nodes From Linked List (Easy)
Remove All Adjacent Duplicates In String (Easy)
☐ Mark as Completed